Contents

Recal

Booleans

Booleans act as conditional values in CS. This is usually described in terms of truth tables and basic operations. The fundamental understanding is that a boolean is either *true* or *false*.

Control Flow

We now explore the process of conditional logic. First we will explore this process of pseudocode rather than python. The concept of condition logic should be rather simple, of course *should* is presumptuous. We can think of the basic examples first then a more advanced type.

If-then-else

We can reference English heavily to understand if conditions. For example: **if** it is *raining* I will bring an *umbrella*, **otherwise** I will not. The condition is began with the beginning characters **if** next is the condition *raining*, then depending on the truth of *raining* we utilize the *umbrella*, the action if raining is false is described after the **otherwise** term. There are many ways to infer conditional logic in a pseudocode like form. The form Ill show is neither the most common nor the most readable, however it is the most clear version in my opinion.

```
if <boolean> then <action on true> else <action on false>
-- WARNING: the above is not runnable code
```

The above code explains the structure of how we create these if blocks.

$$\text{if } a \in \mathbb{Z} \text{ then } a + 1 \text{ else } \sqrt{a} \tag{1}$$

We observe in Eq. 1 the pseudocode structure for a common if statement. This is very much like english, its the *mathy* way to write it. A much more common and equally valid approach (though) not as pretty is to use a more imparative process.

$$\begin{gathered} \text{if } a \in \mathbb{Z} \text{ then} \\ a + 1 \\ \text{else} \\ \sqrt{a} \end{gathered} \tag{2}$$

This way of doing it is essentially the same, simply integrating end line characters for reading purposes[1]. The way this works that is the *condition* value evaluates to a boolean value. its much easier to see in the python code below.

[1] it goes without saying that both styles will get full marks on an exam

```
cond = true
if cond:
    print("the true action")
else:
    print("the false action")
```

Match blocks

If your conditional logic is not linear or branch but rather large then match blocks can be useful. A match block is something that allows one to filter through *many* values all at once.

```
match color:
    case "blue":
        print("we have the blue color")
    case "red":
        print("we have the red color")
    case "green":
        print("we have the green color")
    case _:
        print("unknown color")
```

The match block is comprised of two simple keywords in python: *match*, *case*. It is obvious how to build and use these blocks but there are a few infered rules that need attention. The main rule is that we always need a default case, if we look at the last case it lists the _ value, which of course is the **wildcard** matching any value. The other rule to keep in mind is the evaluated types of the match value and case values need to match.

Python syntax (1)

% How to start a file and compile it

Python syntax can be tricky, understanding a programming language is something that takes time and repetition. The first step in learning any new language is to write the "hello world" program. In python it is as easy as creating a new file, writing `print("hello world")`, and compiling with `pyhton3 <filename>`. Here we will explore the conventional ins and outs of learning the Python language (version 3).

Whats a library?

A Python library is invoked with the `import` keyword. Libraries allow a programmer to include previously written code into their own project. This is useful for things like complex protocols that we as programmers would not want to rewrite, or simply something that is beyond our current ability. In this course we will ignore libraries almost entirely.

Keywords

In Programming languages there exist certain words that are reserved for specific language features. In particular these words are not aloud to appear outside the specified use cases. For example in Python we have already used the `print` keyword, this is a function that sends readable text to the output. Other examples in Python include: `for`, `while`, `if`, `def`, ... etc. Keywords are defined by the compiler for special use only.

Simple use

```
# the octet symbol is a single line comment

x = 1 # This syntax denotes a binding or variable

x = x + 1 # here the value of x will become 2 as the previous value will be used
```

```
# the function f has type f: Int -> Int -> Int
def f(p0,p1):
    return p0 + p1 # the function returns the sum of its parameters

# note the type of function maybe. maybe: a -> a -> Bool -> a
def maybe(p0,p1,p3):
    if p3:
        return p0
    else:
        return p1

# here maybe will receive the output of f(x,2) and the output of f(1,1) separately.
output = maybe(f(x, 2), f(1,1), x > 2) # function invocation
```

Above is a general use case for what we have learned in Python. Below I will give a list of boolean comparison operators, Known as comparison operators.

```
1 < 2 # output True
2 > 2 # outputs false
2 >= 2 # less or equal, outputs True
3 <= 2 # outputs false
2 == 2 # outputs true
```

Nomenclature

Σ l - is the *sumation* of the set l, this means that it takes each element of all and preforms the addition operation on them sequentially
$s.t$ or : - read “such that”
$\forall$ x $\in \mathbb{Z}$ - read “for all x in $\mathbb{Z}$”, denotes iteration over all indices of $\mathbb{Z}$
A $\longleftarrow$ B - denoting a binding where A will represent the value of B
A $\longrightarrow$ B - read A **implies** B, denoting the that the statement A being true implies that B is also true
[ ] - denoting a set
$\mathbb{Z}$ - set of all integers (real numbers or their negatives)
$\mathbb{R}$ - set of all real numbers (rational numbers/irrational numbers, including fractions and their negatives)
i $\in$ I - denotes that i is an element of the set I